Enum

Enum nədir?

C# da enum(nömrələmə),bilinən dəyərləri təmsil etmək üçün istifadə edilən bir tipdir.Enumlar kodumuzu daha anlaşılan və idarə olunan hala gətirir.

```
0 references
public enum Günlər
{
    Bazar,
    BazarErtesi,
    ÇərşənbəAxşamı,
    Çərşənbə,
    CüməAxşamı,
    Cümə,
    Şənbə
}
```

Enum dəyərlərinə necə çatmaq olar?

For dövrü ilə istəfadəsi:

```
Günlər today = Günlər.Cümə;

Console.WriteLine(today); //Cümə
```

```
Günlər today = Günlər.Cümə;

Console.WriteLine((int)today); //5
```

```
Günlər today = (Günlər)5;

Console.WriteLine(today); //Cümə
```

```
foreach (var item in Enum.GetValues(typeof(Günlər)))
{
    Console.WriteLine(item);
}
```

- Default olaraq int tipindən istifadə edir.
- Amma sən byte, short, long və s. də göstərə bilərsən:

```
public enum Status : byte
{
    Pending = 1,
    Approved = 2,
    Rejected = 3
}
```

Qiymət təyin etmə qaydası

1.Əgər heçnə təyin etmirəmsə 0 dan başlayır 1 – 1 artır. Özündə təyin edə bilərsən növbəti elementi +1 artırır.

```
public enum Priority
{
    Low = 5,
    Medium,   // 6
    High      // 7
}
```

Enum-lar və string-lər

Enum.GetName, Enum.GetNames, Enum.GetValues kimi metodlarla adlarını və dəyərlərini ala bilərsən.

```csharp
foreach (var name in Enum.GetNames(typeof(Priority)))
{
    Console.WriteLine(name); // Low, Medium, High
}

foreach (var value in Enum.GetValues(typeof(Priority)))
{
    Console.WriteLine($"{value} = {(int)value}");
}
```

```
Priority p1 = (Priority)Enum.Parse(typeof(Priority), "High");

if (Enum.TryParse("Medium", out Priority p2))
{
    Console.WriteLine(p2); // Medium
}
```

Bu kodun **məqsədi** – **string** kimi gələn bir dəyəri (məsələn istifadəçi inputu, config faylı, API-dən gələn data) C#-dakı **enum** tipinə çevirməkdir. **Enum** tiplə işləmək **int və string-lə işləməkdən daha təhlükəsizdir**.
Kodun daxilində “High”, “Medium” kimi string-lərlə yox, Priority.High, Priority.Medium kimi **mənalı sabitlərlə** işləyirsən. **Niyə Enum.Parse və Enum.TryParse?**
Parse istifadə olunanda string → enum çevrilir. Əgər string yanlışdırsa, exception atır.
TryParse isə daha təhlükəsizdir: əgər string uyğun gəlmirsə exception atmır, sadəcə false qaytarır.
Bu, **istifadəçi daxil etdiyi dəyərin** və ya **serverdən gələn məlumatın** düzgün olub-olmadığını yoxlamaq üçündür.

Tip təhlükəsizliyi (type safety)

•Amma enum dəyişəni **yalnız özündə müəyyən olunmuş dəyərləri** ala bilər.
•Compiler səviyyəsində yoxlama aparılır, səhv dəyər yazanda dərhal build zamanı error verir.

```
public enum Priority { Low, Medium, High }

Priority p = Priority.High; //
p = (Priority)100; // bu cast olmadan olmaz, normalda səhvdir
```

Record nədir?

- C#-da record types siniflər (classes) kimi elan edilir, ancaq class açar sözü əvəzinə record açar sözü istifadə olunur. Record types-in konstruktoru, xüsusiyyətləri (properties) və metodları ola bilir.

```
public record Person(string FirstName, string LastName);
```

Bu yazılış ilə aşağıdakı yazılış bir birinə bərabərdir.

```
public record Person
{
    public required string Name { get; init; }
    public required string LastName { get; init; }
};
```

İnit-Only Properties

- C#9.0-da hər hansı bir obyektin property-lərinə ilk dəyərlərin verilməsi və sonra bu dəyərlərin dəyişdirilməməsini qarantiya altına almağımızı təmin edən **İnit-Only Properties özəlliyi gəlmişdir.**Bu özəllik sayəsində obyektin sadəcə ilk yaradılış anında property-lərinə dəyər verilməkdə və beləliklə runtime-da dəyəri dəyişməməsi lazım olan obyektlər üçün ideal ehtiyat tədbiri görülməkdədir.

Gəlin bir nümunə verək.Tutaq ki bir class-ım var və içərisində bir property var:

```
class MyClass
    {
        public int MyProperty { get; set; }
    }
```

Bu property-ni readonly(getter-only) etmək üçün,set blokunu silmək lazımdır.

Bu property artıq read only-dir.Bu property-ə ilk dəyəri Verək:

```
class MyClass
    {
        public int MyProperty { get; } = 3;
    }
```

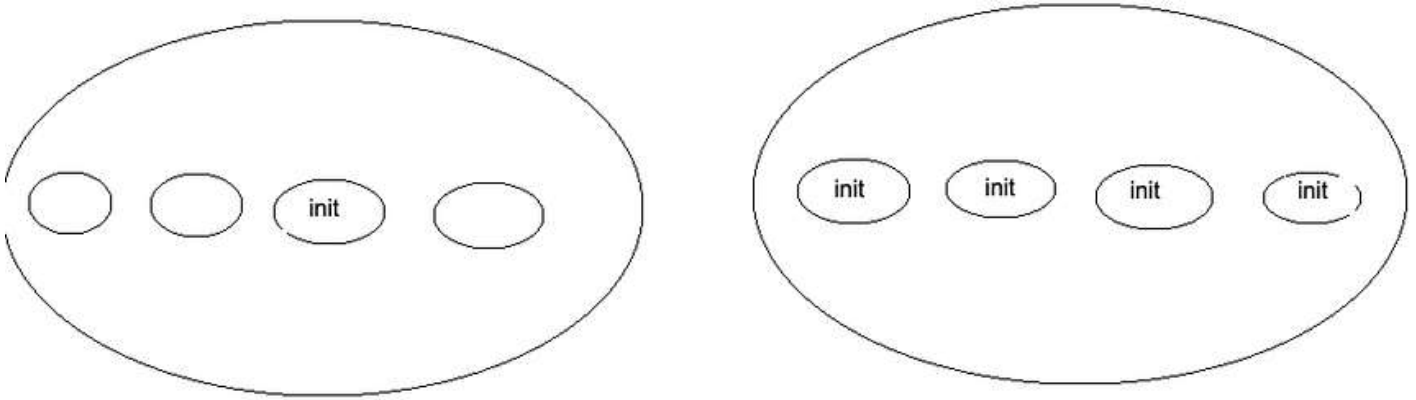
Bildiyimiz kimi,bir porperty readonly-dirsə,həmən sinifdən obyekt yaratsaq object initializer-da ona dəyər verə bilmərik.Tam da bu vəziyyətdə nə istifadə edə bilərik?:Təbii ki,**İnit-Only Properties.**

*Yəni **İnit-Only Properties** ilə Getter-Only-Properties arasındaki fərq odur ki,**İnit-Only properties**-ə object initializer zamanı(obyekt yaradılan anda) dəyər verə bilirik.*

İnit-Only Properties Necə yaradılır?
İnit-Only Properties yaradan zaman ‘init’ keyword-ü bizə kömək edəcək.

```
public string MyProperty { get; init; } = "Məhəmməd";
```

İnit-Only Properties yaradarkən **get** bloku mütləq iştirak etməlidir.Və **set** blokundan istifadə etmək olmaz.



Əgər bir property-nin sabit(readonly) olmağını istəyiriksə **İnit-Only Properties** özəlliyindən istifadə edirik.
Əgər bir obyekti ümumi olaraq dəyişməz hala salmaq istəyiriksə bu zaman daha çoxuna ehtiyacımız var.Yəni obyektin bütün property-lərini **İnit-Only Properties** edirik.Bu ehtiyac üçün **Record** strukturu yaradılmışdır.
Yəni Record bir property-nin dəyişməzliyindən əlavə bir obyektin dəyərinin dəyişməməsini təmin edir.

- Əsas məqsədi immutable (dəyişməz) data obyektlərini rahat modelləşdirməkdir.
- Immutable obyekt – yaradıldıqdan sonra dəyərləri dəyişmir.

```
public record Person(string Name, int Age);

var p = new Person("Valeh", 30);
// p.Age = 31; // Error – init-only
```

“Class-a bənzəyir, amma value-based equality dəstəkləyir (yəni obyektlərdə dəyərlərə görə müqayisə)”

Normalda class obyektləri referansa görə müqayisə olunur.
Amma record dəyərlərə görə müqayisə edir.
Bu value-based equality adlanır. Bu xüsusiyyət sayəsində record-lar çox ideal olur: eyni datanı daşıyan iki obyekt bərabər sayılır.

```
4 references
public class PersonClass
{
    2 references
    public string Name { get; init; }
    2 references
    public int Age { get; init; }
}

4 references
public record PersonRecord
{
    2 references
    public string? Name { get; init; }
    2 references
    public int Age { get; init; }
}
```

```
PersonClass person1 = new PersonClass { Name = "Alice", Age = 30 };
PersonClass person2 = new PersonClass { Name = "Alice", Age = 30 };

Console.WriteLine(person1 == person2); // False


PersonRecord record1 = new PersonRecord { Name = "Bob", Age = 25 };
PersonRecord record2 = new PersonRecord { Name = "Bob", Age = 25 };

Console.WriteLine(record1 == record2); // True
```